

CSCI 112 Assignment 6 – Structs and Function Pointers

Points: 10

Automated tests will run on GitHub Actions after every push. Check the Actions tab to see which tests pass and which fail.

Instructions

1. **Fork** the repository on GitHub - <https://github.com/jpach-cs/112-ass06-template>.
2. Clone your forked repository to your local machine.
3. Open the file `code.h` and add:
 1. Include guards at the top: `#ifndef CODE_H #define CODE_H`
 2. Define structs: `int_Point`, `double_Point`, `Calc` (with function pointers)
 3. Define typedef: `DoublePoint`
 4. All function prototypes
 5. Include guards at the bottom: `#endif`
4. Open the file `code.c`.
 1. Implement every function listed below:
 - Read the comments above each function carefully.
 - Follow the rules — use **ONLY** basic C syntax, no advanced libraries.
 - Function pointers must be used correctly in `apply_operation()` and `init_calc()`.
 2. Test your implementation locally using `main.c`.
5. Once all functions are complete — save your changes, then **commit** and **push** to your repository.
6. Go to the **Actions** tab on GitHub and check the test results.

Learning Objectives

After completing this lab, you will:

- Understand struct initialization
 - Use pointers to structs correctly
 - Implement function pointers and callbacks
 - Pass functions as parameters
 - Use typedef for type aliases
 - Work with nested struct members containing function pointers
-

Implement 6 functions that work with structs and function pointers:

Function	Purpose	Key Concept	Return Type
<code>init_int_point()</code>	Initialize <code>int_Point</code> struct	Struct pointers	<code>void</code>
<code>init_double_point()</code>	Initialize <code>double_Point</code> struct	Struct pointers	<code>void</code>
<code>add()</code>	Add two floats	Basic math	<code>float</code>
<code>sub()</code>	Subtract two floats	Basic math	<code>float</code>
<code>mul()</code>	Multiply two floats	Basic math	<code>float</code>
<code>divide()</code>	Divide two floats	Basic math	<code>float</code>
<code>apply_operation()</code>	Apply function pointer to array	Function pointers (callback)	<code>double</code>
<code>init_calc()</code>	Initialize Calc struct with function pointers	Function pointer assignment	<code>void</code>

Background

Structs and Pointers

When passing structs to functions, use pointers for modification:

```
struct Point { int x; int y; };
```

```
void init_point(struct Point *p, int x, int y) { p->x = x; // Use arrow operator  
-> with pointers p->y = y; }
```

Function Pointers

A function pointer stores the address of a function. Declare it like:

```
float (*operation)(float, float); // Pointer to function taking two floats, return-  
ing float
```

Call the function through the pointer:

```
float result = operation(a, b); // Just like calling a regular function
```

IMPORTANT: When assigning a function to a pointer, use the function name directly (NO &):

```
operation = add; // Correct operation = &add; // Also works but redundant
```

Structs with Function Pointers

You can have struct members that are function pointers:

```
struct Calc { float a; float b; float (add)(float, float); float (sub)(float, float); };  
// Then use it: calc.add(calc.a, calc.b);
```

Typedef

Create type aliases to simplify declarations:

```
typedef struct double_Point { double x; double y; } DoublePoint;  
// Now you can use: DoublePoint p; instead of: struct double_Point p;
```

Function Pointers as Parameters

Pass functions as parameters to implement callbacks:

```
double apply_operation(float arr, int length, float (operation)(float, float)) {  
    float result = arr[0]; for (int i = 1; i < length; i++) { result = operation(result,  
    arr[i]); // Call the passed function } return (double)result; }  
  
// Usage: double sum = apply_operation(arr, 4, add); // Pass add function  
double product = apply_operation(arr, 4, mul); // Pass mul function
```

Only for testing

Local Testing (Sandbox)

Create test cases in main.c:

```
#include <stdio.h>  
#include "code.h"  
  
int main()  
{  
    // Test Step 1: int_Point  
    struct int_Point p1;
```

```

    init_int_point(&p1, 10, 20);
    printf("int_Point: x=%d, y=%d\n", p1.x, p1.y);
    // Test Step 2: double_Point
    struct double_Point p2;
    init_double_point(&p2, 3.14, 2.71);
    printf("double_Point: x=%f, y=%f\n", p2.x, p2.y);

    // Test Step 4: Basic math operations
    printf("add(5, 3) = %f\n", add(5.0f, 3.0f));
    printf("sub(5, 3) = %f\n", sub(5.0f, 3.0f));
    printf("mul(5, 3) = %f\n", mul(5.0f, 3.0f));
    printf("divide(6, 2) = %f\n", divide(6.0f, 2.0f));

    // Test Step 5: apply_operation with function pointers
    float arr[] = {1.0f, 2.0f, 3.0f, 4.0f};
    printf("Sum via apply_operation: %f\n", apply_operation(arr, 4, add));
    printf("Product via apply_operation: %f\n", apply_operation(arr, 4, mul));

    // Test Step 8: Calc struct with function pointers
    struct Calc calc;
    init_calc(&calc);
    calc.a = 10.0f;
    calc.b = 5.0f;
    printf("calc.add(10, 5) = %f\n", calc.add(calc.a, calc.b));
    printf("calc.mul(10, 5) = %f\n", calc.mul(calc.a, calc.b));

    return 0;
}

```